import csv import json import os import sys import hashlib import requests
WEBHOOK_URL = os.getenv("DISPATCH_WEBHOOK_URL")

Set to 'true' if dispatching to a non-BAA channel (e.g. Slack/Teams) to scrub Account IDs

ANONYMIZE_DISPATCH = os.getenv("ANONYMIZE_DISPATCH", "false").lower() == "true"
MAX_ROWS = 20
LEDGER_PATH = "output/sent_ledger.json" QUEUES = [("output/send_sol_notice.csv", "SOL Expungement Notices"), ("output/send_irs_complaint.csv", "501(r) Regulatory Complaints"), ("output/priority_queue.csv", "Certified Mail Buffer Warnings"),]
def load_ledger() -> set: if os.path.exists(LEDGER_PATH): try: with open(LEDGER_PATH, "r", encoding="utf-8") as f: return set(json.load(f)) except Exception as e: print(f"Warning: Could not read ledger ({e}). Starting fresh.", file=sys.stderr) return set()
def save_ledger(ledger: set): os.makedirs("output", exist_ok=True) with open(LEDGER_PATH, "w", encoding="utf-8") as f: json.dump(list(ledger), f, indent=2)
def compute_row_hash(row: dict) -> str: raw_str = f"{row.get('Account ID')}:{row.get('Violation Type')}:{row.get('Action Deadline')}" return hashlib.sha256(raw_str.encode("utf-8")).hexdigest()
def load_new_rows(path: str, ledger: set): if not os.path.isfile(path): return [], []
new_rows = [] new_hashes = []
with open(path, newline="", encoding="utf-8") as f: reader = csv.DictReader(f) for r in reader: h = compute_row_hash(r) if h not in ledger: new_rows.append(r) new_hashes.append(h)
return new_rows, new_hashes
def build_message(label: str, rows: list) -> str: lines = [f"🚨 *{label}*: {len(rows)} new pending work-order(s)"] for r in rows[:MAX_ROWS]: acc_identifier = "[REDACTED-ID]" if ANONYMIZE_DISPATCH else r.get("Account ID", "N/A") doc_ref = r.get("Document ID", "N/A") lines.append(f"- {acc_identifier} | {r.get('Violation Type')} | " f"due: {r.get('Action Deadline')} | ref: {doc_ref}")
if len(rows) > MAX_ROWS: lines.append(f"...and {len(rows) - MAX_ROWS} additional item(s). See build artifact.")
return "\n".join(lines)
def main(): if not WEBHOOK_URL: print("DISPATCH_WEBHOOK_URL not set. Skipping dispatch step.") return 0
ledger = load_ledger() failures = 0 total_dispatched = 0 pending_ledger_updates = []
for path, label in QUEUES: rows, row_hashes = load_new_rows(path, ledger) if not rows: print(f"No new items for {label}.") continue
payload = {"text": build_message(label, rows)}
try: resp = requests.post(WEBHOOK_URL, json=payload, timeout=10) resp.raise_for_status() print(f"Successfully dispatched {len(rows)} new item(s) for {label}.") pending_ledger_updates.extend(row_hashes) total_dispatched += len(rows) except requests.RequestException as exc: failures += 1 print(f"Error dispatching {label}: {exc}", file=sys.stderr)
if pending_ledger_updates: ledger.update(pending_ledger_updates) save_ledger(ledger)

```
if failures > 0: print(f"Dispatch completed with {failures} failure(s).", file=sys.stderr) return 1
print(f"Dispatch pipeline complete. {total_dispatched} total new work-order(s) sent.") return 0
if __name__ == "__main__": sys.exit(main())
```